

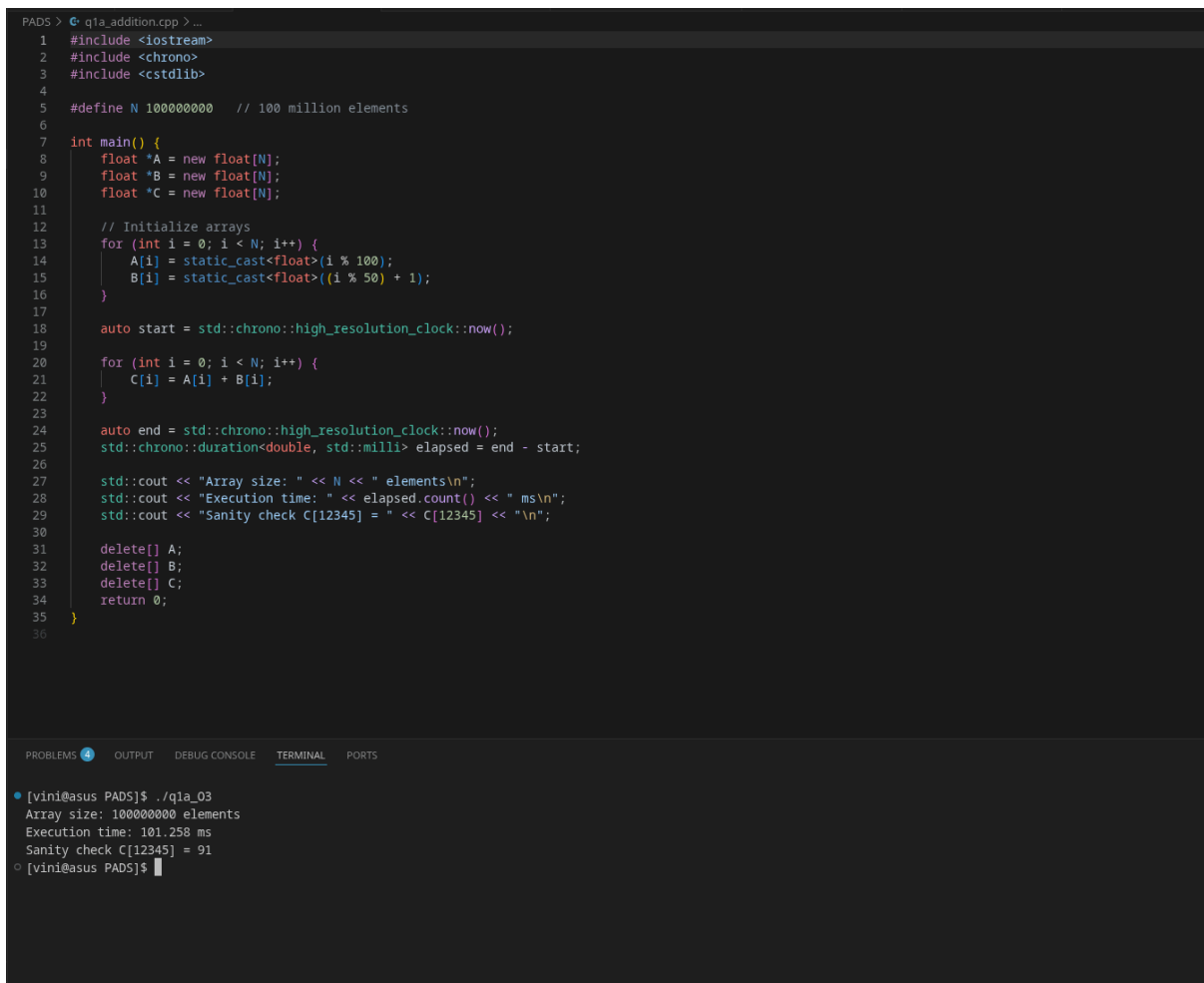
CSS311 – Parallel and Distributed Computing

Lab 3: SIMD, Vector Processing

NAME : VIINEETH MN
ROLLNO : 2024BCS0178

Question 1: SIMD

(a) Write a program to perform element-wise addition of two large arrays. Compile using -O0 and -O3 -march=native. Measure the execution time and compare the results.



```
1 #include <iostream>
2 #include <chrono>
3 #include <cstdlib>
4
5 #define N 100000000 // 100 million elements
6
7 int main() {
8     float *A = new float[N];
9     float *B = new float[N];
10    float *C = new float[N];
11
12    // Initialize arrays
13    for (int i = 0; i < N; i++) {
14        A[i] = static_cast<float>(i % 100);
15        B[i] = static_cast<float>((i % 50) + 1);
16    }
17
18    auto start = std::chrono::high_resolution_clock::now();
19
20    for (int i = 0; i < N; i++) {
21        C[i] = A[i] + B[i];
22    }
23
24    auto end = std::chrono::high_resolution_clock::now();
25    std::chrono::duration<double, std::milli> elapsed = end - start;
26
27    std::cout << "Array size: " << N << " elements\n";
28    std::cout << "Execution time: " << elapsed.count() << " ms\n";
29    std::cout << "Sanity check C[12345] = " << C[12345] << "\n";
30
31    delete[] A;
32    delete[] B;
33    delete[] C;
34    return 0;
35 }
36
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[vini@asus PADS]$ ./q1a_03
Array size: 100000000 elements
Execution time: 101.258 ms
Sanity check C[12345] = 91
[vini@asus PADS]$
```

Sc - program and output of optimized vectorized execution

(c) Modify the loop to introduce a loop-carried dependency (e.g., $A[i] = A[i-1] + B[i]$).
Recompile and analyse why vectorization fails.

```

PADS > g++ q1c_dependency.cpp ...
1  #include <iostream>
2  #include <chrono>
3
4  #define N 100000000
5
6  int main() {
7      float *A = new float[N];
8      float *B = new float[N];
9
10     A[0] = 1.0f;
11     for (int i = 0; i < N; i++) {
12         B[i] = static_cast<float>((i % 50) + 1);
13     }
14
15     auto start = std::chrono::high_resolution_clock::now();
16
17     // Loop-carried dependency: A[i] depends on A[i-1], computed in the
18     // PREVIOUS iteration. Each iteration cannot begin until the previous
19     // one has completed and written its result, so the compiler cannot
20     // pack multiple iterations into one SIMD instruction.
21     for (int i = 1; i < N; i++) {
22         A[i] = A[i - 1] + B[i];
23     }
24
25     auto end = std::chrono::high_resolution_clock::now();
26     std::chrono::duration<double, std::milli> elapsed = end - start;
27
28     std::cout << "Execution time (dependency loop): " << elapsed.count() << " ms\n";
29     std::cout << "Sanity check A[100] = " << A[100] << "\n";
30
31     delete[] A;
32     delete[] B;
33 }

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
[v@linus PADS]$ cd /home/vallu/Documents/DEMO/PADS/ && g++ q1c_dependency.cpp -O q1c_dependency.o /home/vallu/Documents/DEMO/PADS/ q1c_dependency.o
Sanity check A[100] = 2551
[v@linus PADS]$ g++ -O3 -march=native -fopenmp -fopenmp-libs -fopenmp-libs -fopenmp-libs -fopenmp-libs q1c_dependency.cpp -O q1c_dependency.o
./q1c_03
q1c_dependency.cpp:27:23: missed: couldn't vectorize loop
q1c_dependency.cpp:28:23: missed: not vectorized: no vectype for stmt: _10 = *_9;
  scalar_type: float
q1c_dependency.cpp:17:23: optimized: loop vectorized using 32 byte vectors and unroll factor 8
q1c_dependency.cpp:13:27: missed: statement clobbers memory: _23 = operator new [] (400000000);
q1c_dependency.cpp:14:27: missed: statement clobbers memory: _26 = operator new [] (400000000);
q1c_dependency.cpp:21:57: missed: statement clobbers memory: start = std::chrono::_V2::system_clock::now ();
q1c_dependency.cpp:31:55: missed: statement clobbers memory: end = std::chrono::_V2::system_clock::now ();
/usr/include/c++/16/bits/ostream.h:739:18: missed: statement clobbers memory: std::_ostream_insert<char, std::char_traits<char> > (&cout, "Execution time (dependency loop)
/usr/include/c++/16/bits/ostream.h:732:25: missed: statement clobbers memory: _48 = std::basic_ostream<char>::_M_insert<double> (&cout, _44);
/usr/include/c++/16/bits/ostream.h:739:18: missed: statement clobbers memory: std::_ostream_insert<char, std::char_traits<char> > (&cout, "ms\n", 4);
/usr/include/c++/16/bits/ostream.h:739:18: missed: statement clobbers memory: std::_ostream_insert<char, std::char_traits<char> > (&cout, "Sanity check A[100] = ", 22);
/usr/include/c++/16/bits/ostream.h:739:18: missed: statement clobbers memory: _50 = std::basic_ostream<char>::_M_insert<double> (&cout, _d_49);
/usr/include/c++/16/bits/ostream.h:739:18: missed: statement clobbers memory: std::_ostream_insert<char, std::char_traits<char> > (&cout, "\n", 1);
q1c_dependency.cpp:37:14: missed: statement clobbers memory: operator delete [] (_23);
q1c_dependency.cpp:38:14: missed: statement clobbers memory: operator delete [] (_26);
Execution time (dependency loop): 132.19 ms
Sanity check A[100] = 2551

```

Sc - program and output

Reason why vectorization fails

Vectorization fails because $A[i]$ depends on $A[i-1]$ computed in the previous iteration (a loop-carried dependency), so the iterations cannot be executed independently/simultaneously as required by SIMD; the compiler must fall back to strictly sequential scalar execution to preserve correctness.

(d) Implement the same program using OpenMP only, and OpenMP+ SIMD optimization.
Measure the execution time and compare the results.

```
PADS > q1d_openmp_simd.cpp > ...
1 // Question 1(d): Same addition using OpenMP + SIMD directive combined
2 #include <iostream>
3 #include <chrono>
4 #include <omp.h>
5
6 #define N 100000000
7
8 int main() {
9     float *A = new float[N];
10    float *B = new float[N];
11    float *C = new float[N];
12
13    for (int i = 0; i < N; i++) {
14        A[i] = static_cast<float>(i % 100);
15        B[i] = static_cast<float>((i % 50) + 1);
16    }
17
18    auto start = std::chrono::high_resolution_clock::now();
19
20    // parallel for splits iterations across threads;
21    // simd tells each thread to also vectorize its chunk of the loop
22    #pragma omp parallel for simd
23    for (int i = 0; i < N; i++) {
24        C[i] = A[i] + B[i];
25    }
26
27    auto end = std::chrono::high_resolution_clock::now();
28    std::chrono::duration<double, std::milli> elapsed = end - start;
29
30    std::cout << "Threads used: " << omp_get_max_threads() << "\n";
31    std::cout << "Execution time (OpenMP + SIMD): " << elapsed.count() << " ms\n";
32    std::cout << "Sanity check C[12345] = " << C[12345] << "\n";
33
34    delete[] A;
35    delete[] B;
36    delete[] C;
37    return 0;
38 }
39
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[vini@asus PADS]$ g++ -O3 -march=native -fopenmp q1d_openmp_simd.cpp -o q1d_openmp_simd
[vini@asus PADS]$ ./q1d_openmp_simd
Threads used: 16
Execution time (OpenMP + SIMD): 77.2089 ms
Sanity check C[12345] = 91
[vini@asus PADS]$
```

Sc - same addition using OpenMP + SIMD directive combined(program and output)

```
PADS > q1d.openmp.cpp > ...
1 // Question 1(d): Same addition using OpenMP only (multi-threaded, no explicit SIMD)
2 #include <iostream>
3 #include <chrono>
4 #include <omp.h>
5
6 #define N 100000000
7
8 int main() {
9     float *A = new float[N];
10    float *B = new float[N];
11    float *C = new float[N];
12
13    for (int i = 0; i < N; i++) {
14        A[i] = static_cast<float>(i % 100);
15        B[i] = static_cast<float>((i % 50) + 1);
16    }
17
18    auto start = std::chrono::high_resolution_clock::now();
19
20    #pragma omp parallel for
21    for (int i = 0; i < N; i++) {
22        C[i] = A[i] + B[i];
23    }
24
25    auto end = std::chrono::high_resolution_clock::now();
26    std::chrono::duration<double, std::milli> elapsed = end - start;
27
28    std::cout << "Threads used: " << omp_get_max_threads() << "\n";
29    std::cout << "Execution time (OpenMP only): " << elapsed.count() << " ms\n";
30    std::cout << "Sanity check C[12345] = " << C[12345] << "\n";
31
32    delete[] A;
33    delete[] B;
34    delete[] C;
35    return 0;
36 }
37
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
[vini@asus PADS]$ g++ -O3 -fopenmp q1d.openmp.cpp -o q1d_openmp
[vini@asus PADS]$ ./q1d_openmp
Threads used: 16
Execution time (OpenMP only): 73.6322 ms
Sanity check C[12345] = 91
[vini@asus PADS]$
```

Sc- same addition using OpenMP multithreaded no explicit SIMD(program and output)

Result comparison

OpenMP+SIMD showed no additional speedup over OpenMP alone because the addition kernel is memory-bandwidth-bound once parallelized across 16 threads; the arithmetic intensity (1 FLOP per 12 bytes moved) is too low for SIMD's extra ALU throughput to matter here."